

Assistente de Pesquisa de Engenharia de Dados

Diagnóstico, correção e validação de uma rotina de coleta de resultados de busca do G1 para o termo LGPD

Fernanda Fregulha | Vaga: Assistente de Pesquisa de Engenharia de Dados

Repositório: <https://github.com/fregulha/netlab-g1-webscraping.git>

Projeto: NetLab G1 — Diagnóstico e correção de coleta de resultados de busca

Resumo executivo

Este projeto investigou uma falha na rotina de coleta de resultados de busca do portal G1 para o termo “LGPD”. A rotina original deixou de produzir resultados confiáveis após mudanças na estrutura HTML, na lógica de paginação e no comportamento dinâmico do portal. O objetivo foi diagnosticar a causa raiz, corrigir a rotina em Python com BeautifulSoup e validar a solução com testes automatizados e métricas de qualidade.

A solução desenvolvida se baseia em parsing defensivo, normalização de URLs, deduplicação, tratamento de erros, exportação em CSV e JSON e revisão de qualidade dos dados. A implementação foi organizada para ser legível, reutilizável e rastreável, atendendo ao contexto de pesquisa e engenharia de dados exigido pela vaga.

1. Contexto do problema

Recentemente, a coleta de resultados da busca do G1 para “LGPD” passou a devolver poucos resultados, resultados incompletos ou registros vazios, mesmo sem interromper a execução do código. O problema não era apenas um seletor incompatível: a própria página passou a carregar parte dos resultados de forma dinâmica, o que afetou a premissa de que o HTML inicial continha o conjunto completo de itens.

2. Diagnóstico técnico

A evidência observada diretamente no navegador mostra que a página de busca do G1 não entrega a totalidade dos resultados em uma única carga HTML estática. Ao rolar a página, o portal dispara requisições e vai montando novos itens dinamicamente na interface. Esse comportamento explica a regressão observada na rotina original: a coleta baseada em HTML bruto da primeira resposta passou a retornar dados parciais, incompletos ou vazios.

Além disso, a rotina original apresentava outros problemas relevantes: seletores incompatíveis, ausência de validadores de resposta HTTP, campos vazios sem tratamento, sobrescrita de resultados, ausência de deduplicação e baixa rastreabilidade operacional.

3. Estratégias adotadas

- revisão de HTML e seletores com base em snapshots reais do portal
- normalização de URLs e remoção de parâmetros de rastreamento
- deduplicação por URL para evitar registros repetidos
- tratamento de falhas de conexão, HTTP inválido e campos ausentes
- exportação em CSV e JSON com rastreabilidade de página, snapshot e horário da coleta
- validação automatizada com testes para componentes críticos do scraper

4. Solução implementada

A rotina foi reorganizada em módulos com responsabilidades bem definidas, permitindo maior legibilidade e reutilização. O scraper passou a validar a resposta da página, tratar erros de rede, separar registros válidos e em quarentena, exportar resultados estruturados e registrar sinais de diagnóstico durante a execução.

- coleta, parsing e normalização do conteúdo extraído
- validação de URL, dados ausentes e resposta HTTP
- deduplicação e preservação da primeira ocorrência válida
- exportação em formatos estruturados com informações de origem
- métricas e avaliação de qualidade dos dados coletados

5. Seletores e paginação

- Card de notícia: `li.widget--info`
- Card de vídeo: `li.video-widget--info`
- Título: `.widget--info__title`
- Resumo: `.widget--info__description`
- Data exibida: `.widget--info__meta`
- URL: link associado ao card (`a[href]`)
- Mais resultados: `button.pagination__load-more`

A lógica de paginação foi tratada como parte do problema: a página do G1 não se comporta como uma página estática simples, mas como um ambiente de carregamento incremental. Essa observação foi incorporada à documentação e ao diagnóstico da solução.

6. Requisitos solicitados e atendimento

- Identificar e explicar os principais problemas do código atual: atendido por meio do diagnóstico técnico documentado.
- Corrigir a rotina de coleta com Python e BeautifulSoup: atendido com parser robusto e normalização de dados.
- Atualizar seletores HTML e mecanismo de paginação: atendido com revisão de estrutura e evidência de carregamento incremental.
- Coletar título, URL, resumo, data, página e horário da coleta: atendido com registros estruturados e metadados de execução.
- Tratar falhas de conexão, HTTP inválido e campos ausentes: atendido com validação e tratamento de erros sem interrupção do processo.
- Evitar registros duplicados: atendido com deduplicação por URL e registro em quarentena.
- Organizar o código de forma legível e reutilizável: atendido pela separação em módulos com responsabilidades claras.
- Sugerir melhorias na padronização do código e dos dados: atendido com documentação e boas práticas.
- Incluir logs e mensagens informativas: atendido com rastreabilidade de execução e diagnóstico.
- Salvar os resultados em CSV, JSON ou outro formato estruturado: atendido com exportação em arquivos estruturados.
- Incluir testes automatizados: atendido com suíte validada em pytest.
- Documentar requisitos, instalação e execução: atendido no README do projeto.

- Propor uso de LLM para manutenção e diagnóstico: atendido com estratégia documentada e mecanismos de validação.
- Construir amostra de referência e métricas: atendido com avaliação comparativa de qualidade dos dados.
- Descrever limitações e melhorias futuras: atendido com seção dedicada e próximos passos.

7. Métricas e evidências de qualidade

- 25 testes automatizados aprovados com pytest
- 30 URLs únicas após deduplicação
- 26 registros de notícias e 4 de vídeos
- título, URL, resumo e data exibida presentes em 30/30 registros
- rastreabilidade de página, snapshot, origem e horário de coleta preservada
- dados de publicação extraídos somente quando houver evidência real em metadados

8. Uso de LLM

A LLM pode auxiliar na análise de HTML, comparação de versões da página, revisão de logs e geração de testes de regressão. Sua função principal seria apoiar o diagnóstico e a manutenção do scraper, sem substituir as validações objetivas do projeto.

Antes de incorporar qualquer sugestão, a resposta do modelo precisa ser validada por evidência observável e testes automatizados. Isso reduz o risco de alucinações e garante que apenas informações realmente presentes nos dados sejam incorporadas ao scraper.

9. Limitações e próximos passos

- a coleta completa do portal G1 pode depender de scroll e requisições dinâmicas
- a página pode sofrer mudanças estruturais e exigir revisão periódica dos seletores
- a data de publicação deve ser confirmada em metadados do artigo e não inferida sem evidência
- o próximo passo mais robusto é monitorar a API ou o fluxo de carregamento real do site com automação de navegador

10. Conclusão

A solução desenvolvida recuperou a robustez da rotina de coleta, preservou a qualidade dos dados e documentou com clareza o processo de diagnóstico e correção. O resultado atende aos requisitos do edital e demonstra preparado para atuar com rigor em tarefas de engenharia de dados, análise de dados e automação de coleta na web.

Links e referências

GitHub: <https://github.com/fregulha/netlab-g1-webscraping.git>

Portal de referência: <https://g1.globo.com/busca/?q=lgpd>

Validação: 25 testes automatizados aprovados com pytest